KAMMAVARI SANGHAM INSTITUTE OF TECHNOLOGY

(Affiliated to Visvesvaraya Technological University, Belagavi | Approved by AICTE, New Delhi)

Department of Computer Science & Engineering

Bengaluru – 560109, Karnataka, India

Cryptography and Network Security

Course Code: CNS | Academic Year: 2026

Comprehensive Laboratory & Technical Audit Report

Assignment Title:

Implementation, Empirical Verification, and Cryptanalytic Evaluation of Classical and Modern Cryptographic Systems

Batch Identifier: `2026_CNS_A_01`

Activity Allocation: Classical Substitution (CO1), Polyalphabetic Substitution (CO4), Information-Theoretic Secrecy (CO3), and Digital Signatures (CO3)

Sl. No.	USN	Student Name	Designated Laboratory Area
1	`1KS23CS059`	**Jayaditya Dev**	Unified TUI Engine & Activity 4 (RSA-PSS)
2	`1KS23CS043`	**Durgashree M**	Activity 3 (One-Time Pad CSPRNG)
3	`1KS23CS020`	**Aryaman Tiwari**	Activity 2 (Vigenère Tabula Recta)
4	`1KS23CS034`	**Chirag T**	Activity 1 (Caesar Modular Cipher)

Submission Date: Academic Term 2026

Evaluation Scheme: Total 25 Marks (Part A Demonstration: 15 Marks | Part B Technical Report: 10 Marks)

Contents

1 Abstract **1**

2 Introduction and Objectives **1**
2.1 Background and Cryptographic Problem 1
2.2 Course Outcome Mapping and Specific Objectives 1

3 Theoretical Background **2**
3.1 Activity 1: Caesar Cipher and the Ring $\mathbb{Z}_{26}$ 2
3.2 Activity 2: Vigenère Cipher and Polyalphabetic Mapping 2
3.3 Activity 3: One-Time Pad and Shannon's Perfect Secrecy 2
3.4 Activity 4: Digital Signatures and RSA-PSS 3

4 Working Principles and Mathematical Traces **3**
4.1 Trace 1: Caesar Cipher Encryption and Decryption 3
4.2 Trace 2: Vigenère Polyalphabetic Stream Alignment 4
4.3 Trace 3: One-Time Pad Bytewise XOR Transformation 4
4.4 Trace 4: RSA-PSS Signing and Verification 4

5 Algorithms, Pseudocode, and Flowcharts **5**
5.1 Algorithmic Pseudocode 5
5.2 Architectural Flowcharts 5

6 System Requirements and Implementation Environment **6**
6.1 Hardware and Host Operating Environment 6
6.2 Software Toolchain and Verified Dependencies 6
6.3 Repository Architecture and Modular Packaging 7

7 Implementation Details **8**
7.1 Activity 1: Caesar Cipher (`activity1_caesar/caesar.py`) 8
7.2 Activity 2: Vigenère Cipher (`activity2_vigenere/vigenere.py`) 8
7.3 Activity 3: One-Time Pad (`activity3_one_time`$_{pad/otp.py}$) 8
7.4 Activity 4: Digital Signature (`activity4_digital_signature/signature.py`) 8
7.5 Unified Terminal User Interface (`main.py`) 9

8 Execution Procedure and Demonstration Transcripts **9**
8.1 Execution Instructions from a Clean Environment 9
8.2 Genuine Terminal Demonstration Outputs 9
8.2.1 Activity 1: Caesar Cipher Execution 9
8.2.2 Activity 2: Vigenère Cipher Execution 9
8.2.3 Activity 3: One-Time Pad & Catastrophic Key-Reuse Attack 10
8.2.4 Activity 4: Digital Signature Tamper & Key-Mismatch Detection 10

9 Testing Methodology and Results **10**
9.1 Testing Approach and Test Categories 10
9.2 Exhaustive 69-Test Verification Matrix 11

10 Security Analysis and Cryptanalysis **12**
10.1 Monoalphabetic Cryptanalysis (Caesar Cipher) 12
10.2 Polyalphabetic Cryptanalysis (Vigenère Cipher) 13
10.3 Information-Theoretic Analysis (One-Time Pad) 13
10.3.1 The Five Mandatory Conditions for Shannon's Perfect Secrecy 13

10.3.2 Catastrophic Two-Time Pad Vulnerability 14
10.4 Asymmetric Security Analysis (RSA-PSS) 14
10.4.1 Security Properties Provided 14
10.4.2 RSA-PSS vs. Legacy PKCS#1 v1.5 Padding 14
10.4.3 Digital Signatures vs. Encryption and the Role of PKI 14

11 Advantages and Limitations **14**

12 Practical and Historical Applications **14**
12.1 Historical Applications 14
12.2 Modern Industry Applications 15

13 Conclusion **15**

14 Group Contribution Matrix **17**

1 Abstract

This technical report delivers a rigorous, empirical investigation and software audit of four fundamental cryptographic paradigms implemented for the Cryptography and Network Security course under Batch allocation `2026_CNS_A_01`. The investigated systems span: (1) classical monoalphabetic substitution (Caesar Cipher), (2) polyalphabetic keyword substitution (Vigenère Cipher), (3) information-theoretic symmetric stream encryption (One-Time Pad), and (4) industry-standard asymmetric authentication and integrity verification (RSA-PSS digital signatures with SHA-256).

Every cryptographic primitive was implemented natively in Python 3.14 without relying on high-level cryptographic abstractions for classical ciphers, while employing the rigorously audited, PyCA-backed `cryptography` engine for asymmetric primitives. All algorithms were subjected to automated verification comprising 69 distinct test cases spanning deterministic Known-Answer Tests (KAT), cyclic boundary conditions, malformed input handling, and negative tamper detection. All 69 tests executed successfully in 0.55 seconds with zero defects. Beyond functional implementation, this report provides formal cryptanalytic evaluations: exhaustive search analysis, frequency analysis, Kasiski examination, Friedman's Index of Coincidence, Shannon's unconditional secrecy theorem, the catastrophic Two-Time Pad vulnerability, and the architectural distinction between raw mathematical key pairs and Public Key Infrastructure (PKI). Genuine terminal transcripts and pipeline data flows establish that modern security requires provable cryptographic primitives rather than security through obscurity.

2 Introduction and Objectives

2.1 Background and Cryptographic Problem

Data security in computer networks rests on four core pillars: confidentiality, integrity, authenticity, and non-repudiation. Historically, cryptographic systems sought confidentiality through character rearrangement or substitution. However, classical ciphers suffer from algebraic structure preservation and statistical predictability, allowing adversaries to recover secret messages with minimal effort. The transition to modern symmetric cryptography introduced information-theoretic secrecy, while the emergence of asymmetric cryptography solved the longstanding key-distribution paradox and introduced digital signatures capable of binding message origin to a verifiable identity.

2.2 Course Outcome Mapping and Specific Objectives

In accordance with the official course syllabus and topic distribution for Batch `2026_CNS_A_01`, this work addresses three designated Course Outcomes (COs):

1. **CO1 – Classical Substitution Ciphers (Activity 1)**: Formulate, implement, and analyze monoalphabetic Caesar shifts; demonstrate modular normalization across arbitrary integer keys; evaluate vulnerability to exhaustive key search and unigram frequency analysis.

2. **CO4 – Polyalphabetic and Matrix-Based Ciphers (Activity 2)**: Formulate and implement the Vigenère cipher using the 26×26 Tabula Recta; enforce strict key-stream advancement rules over non-alphabetic tokens; model cryptanalytic breakdown via Kasiski examination and Friedman's Index of Coincidence.

3. **CO3 – Information-Theoretic Secrecy (Activity 3)**: Construct a bytewise One-Time Pad (OTP) using a Cryptographically Secure Pseudo-Random Number Generator (CSPRNG); prove Claude Shannon's 1949 theorem of perfect secrecy; demonstrate the empirical collapse of secrecy under key reuse (Two-Time Pad attack).

4. **CO3 – Asymmetric Cryptography and Integrity (Activity 4)**: Implement a production-grade digital signature system using 2048-bit RSA keys and Probabilistic Signature Scheme

(PSS) padding with SHA-256; demonstrate tamper detection; contrast mathematical signatures with Public Key Infrastructure (PKI).

3 Theoretical Background

3.1 Activity 1: Caesar Cipher and the Ring $\mathbb{Z}_{26}$

The Caesar cipher is an additive monoalphabetic substitution cipher defined over the finite residue ring of integers modulo 26:

$$\mathbb{Z}_{26} = \{0, 1, 2, \ldots, 25\} \tag{1}$$

The 26 uppercase Latin letters are mapped injectively onto $\mathbb{Z}_{26}$ via the bijection $\pi : \mathcal{A} \to \mathbb{Z}_{26}$ where $\pi(\text{'A'}) = 0, \pi(\text{'B'}) = 1, \ldots, \pi(\text{'Z'}) = 25$. For any integer key $K \in \mathbb{Z}$, the encryption map $f_K : \mathbb{Z}_{26} \to \mathbb{Z}_{26}$ is defined by:

$$C_i = f_K(P_i) = (P_i + K) \pmod{26} \tag{2}$$

Because addition in $\mathbb{Z}_{26}$ forms an abelian group, the decryption map is the additive inverse:

$$P_i = f_K^{-1}(C_i) = (C_i - K) \pmod{26} = (C_i + (-K \pmod{26})) \pmod{26} \tag{3}$$

The Caesar cipher preserves the underlying letter frequencies of natural language because the permutation applied to each alphabet symbol is static throughout the plaintext.

3.2 Activity 2: Vigenère Cipher and Polyalphabetic Mapping

The Vigenère cipher addresses the frequency preservation of monoalphabetic ciphers by using polyalphabetic substitution. Instead of a single static shift, a sequence of shifts derived from an alphabetic keyword $K = (k_0, k_1, \ldots, k_{L-1})$ of length L is applied cyclically:

$$C_i = (P_i + k_{i \pmod{L}}) \pmod{26} \tag{4}$$

Decryption shifts backward using the matching key character:

$$P_i = (C_i - k_{i \pmod{L}}) \pmod{26} \tag{5}$$

This operation corresponds to looking up entries in a 26×26 grid called the **Tabula Recta**, where row P_i and column k_j intersect at character C_i. By cycling through L distinct alphabets, the unigram frequency distribution of the plaintext is smoothed across the ciphertext.

3.3 Activity 3: One-Time Pad and Shannon's Perfect Secrecy

In 1949, Claude Shannon established the mathematical foundations of cryptography in his seminal paper *"Communication Theory of Secrecy Systems"* [1]. Shannon defined **perfect secrecy** (information-theoretic or unconditional secrecy) as the condition where the a posteriori probability distribution of the plaintext given the ciphertext is identical to its a priori probability distribution:

$$\mathbb{P}(M = m \mid C = c) = \mathbb{P}(M = m) \quad \forall m \in \mathcal{M}, c \in \mathcal{C} \tag{6}$$

Equivalently, expressed in terms of Shannon entropy $H(M)$:

$$H(M \mid C) = H(M) \tag{7}$$

Shannon proved that for a cipher to achieve perfect secrecy:

$$H(K) \geq H(M) \implies |\mathcal{K}| \geq |\mathcal{M}| \tag{8}$$

The One-Time Pad operates over byte vectors $M, K \in \{0,1\}^8$ using bitwise Exclusive-OR (addition modulo 2):

$$C = M \oplus K, \quad M' = C \oplus K = (M \oplus K) \oplus K = M \oplus (K \oplus K) = M \oplus \mathbf{0} = M \tag{9}$$

For any observed ciphertext byte c and any candidate plaintext byte m^*, there exists exactly one key $k^* = c \oplus m^*$. If K is uniformly distributed over $\{0,1\}^8$, then $\mathbb{P}(K = k^*) = 1/256$, rendering every plaintext equally probable regardless of adversarial computing capacity.

3.4 Activity 4: Digital Signatures and RSA-PSS

In asymmetric cryptography, a digital signature provides authenticity, message integrity, and non-repudiation. Unlike symmetric message authentication codes (MACs), digital signatures are publicly verifiable without revealing the signing secret.

An RSA key pair consists of a public key (e, n) and a private key (d, n) where $n = p \cdot q$ for large primes p, q, and $e \cdot d \equiv 1 \pmod{\phi(n)}$. To prevent homomorphic attacks and chosen-ciphertext vulnerabilities present in textbook RSA ($S = h^d \pmod n$), the **Probabilistic Signature Scheme (PSS)** (RFC 8017 / PKCS#1 v2.1 [4]) is utilized:

1. **Hashing**: Message M is hashed using SHA-256 to produce fixed digest $h = H(M) \in \{0,1\}^{256}$.
2. **Encoding**: h is combined with a cryptographically generated random salt of length s_{len} through Mask Generation Function 1 (MGF1) to form encoded message representative $\mu \in \mathbb{Z}_n$.
3. **Exponentiation**: The signer computes signature S:
$$S = \mu^d \pmod n \tag{10}$$
4. **Verification**: The verifier computes $\mu' = S^e \pmod n$, unpacks the PSS structure, and verifies that the extracted digest matches $H(M)$.

4 Working Principles and Mathematical Traces

4.1 Trace 1: Caesar Cipher Encryption and Decryption

Input Vector: Plaintext $P =$ `"CNS 2026"`, Key $K = 29$.

1. **Key Normalization**:
$$K_{\text{norm}} = 29 \pmod{26} = 3 \tag{11}$$
2. **Character-by-Character Transformation**:
   - $P_0 =$ 'C' $\implies \pi('C') = 2$. $C_0 = (2+3) \pmod{26} = 5 \implies$ 'F'.
   - $P_1 =$ 'N' $\implies \pi('N') = 13$. $C_1 = (13+3) \pmod{26} = 16 \implies$ 'Q'.
   - $P_2 =$ 'S' $\implies \pi('S') = 18$. $C_2 = (18+3) \pmod{26} = 21 \implies$ 'V'.
   - $P_3 =$ ' ', $P_4 =$ '2', $P_5 =$ '0', $P_6 =$ '2', $P_7 =$ '6'. All are non-alphabetic; preserved verbatim.

   Generated Ciphertext: `"FQV 2026"`.
3. **Decryption**: Apply shift $-3 \pmod{26} = 23$:
   - $C_0 =$ 'F' $\implies (5+23) \pmod{26} = 28 \pmod{26} = 2 \implies$ 'C'.

   Recovered Plaintext: `"CNS 2026"` (Verified identical).

4.2 Trace 2: Vigenère Polyalphabetic Stream Alignment

Input Vector: Plaintext P = `"ATTACK AT DAWN!"`, Keyword K = `"LEMON"`. The normalized keyword length is $L = 5$. The keyword shifts are: $\mathrm{L} = 11, \mathrm{E} = 4, \mathrm{M} = 12, \mathrm{O} = 14, \mathrm{N} = 13$. **Ciphertext Output**: `"LXFOPV EF RNHR!"`. Notice how the two 'A's at indices 0 and 7 map to

Index i	Char P_i	Key Char	Shift k_j	Formula $(P_i + k_j) \pmod{26}$	Result C_i
0	'A' (0)	'L'	11	$(0 + 11) \pmod{26} = 11$	'L'
1	'T' (19)	'E'	4	$(19 + 4) \pmod{26} = 23$	'X'
2	'T' (19)	'M'	12	$(19 + 12) \pmod{26} = 5$	'F'
3	'A' (0)	'O'	14	$(0 + 14) \pmod{26} = 14$	'O'
4	'C' (2)	'N'	13	$(2 + 13) \pmod{26} = 15$	'P'
5	'K' (10)	'L'	11	$(10 + 11) \pmod{26} = 21$	'V'
6	' '	–	–	*Whitespace preserved; key paused*	' '
7	'A' (0)	'E'	4	$(0 + 4) \pmod{26} = 4$	'E'
8	'T' (19)	'M'	12	$(19 + 12) \pmod{26} = 5$	'F'
9	' '	–	–	*Whitespace preserved; key paused*	' '
10	'D' (3)	'O'	14	$(3 + 14) \pmod{26} = 17$	'R'
11	'A' (0)	'N'	13	$(0 + 13) \pmod{26} = 13$	'N'
12	'W' (22)	'L'	11	$(22 + 11) \pmod{26} = 7$	'H'
13	'N' (13)	'E'	4	$(13 + 4) \pmod{26} = 17$	'R'
14	'!'	–	–	*Punctuation preserved verbatim*	'!'

'L' and 'E' respectively, demonstrating polyalphabetic flattening.

4.3 Trace 3: One-Time Pad Bytewise XOR Transformation

Input Vector: Plaintext M = `"CNS 2026"` (8 bytes UTF-8: `0x43 0x4E 0x53 0x20 0x32 0x30 0x32 0x36`).
Generated CSPRNG Key K: `0x9A 0x2B 0x7F 0xE1 0x0C 0x88 0x64 0x19` (8 bytes). **Ci-**

Byte #	Char	Plaintext Byte	CSPRNG Key Byte	Bitwise XOR	Ciphertext Byte
1	'C'	`0x43` (01000011)	`0x9A` (10011010)	01000011 ⊕ 10011010	`0xD9` (11011001)
2	'N'	`0x4E` (01001110)	`0x2B` (00101011)	01001110 ⊕ 00101011	`0x65` (01100101)
3	'S'	`0x53` (01010011)	`0x7F` (01111111)	01010011 ⊕ 01111111	`0x2C` (00101100)
4	' '	`0x20` (00100000)	`0xE1` (11100001)	00100000 ⊕ 11100001	`0xC1` (11000001)
5	'2'	`0x32` (00110010)	`0x0C` (00001100)	00110010 ⊕ 00001100	`0x3E` (00111110)
6	'0'	`0x30` (00110000)	`0x88` (10001000)	00110000 ⊕ 10001000	`0xB8` (10111000)
7	'2'	`0x32` (00110010)	`0x64` (01100100)	00110010 ⊕ 01100100	`0x56` (01010110)
8	'6'	`0x36` (00110110)	`0x19` (00011001)	00110110 ⊕ 00011001	`0x2F` (00101111)

phertext Hex: `D9652CC13E88562F`. Decryption: $C \oplus K$ yields `0x43 0x4E 0x53 0x20 0x32 0x30 0x32 0x36`, successfully decoded as `"CNS 2026"`.

4.4 Trace 4: RSA-PSS Signing and Verification

Execution Workflow:

1. **Key Generation**: Generate 2048-bit modulus n, public exponent $e = 65537$, and private exponent d.
2. **Input Message**: `"Confidential Academic Dispatch 2026"`.
3. **Digest Generation**:

$$h = \text{SHA-256}(M) = \texttt{b8c8d23e5932a39c4d96a77d...} \quad \text{(32 bytes / 256 bits)} \tag{12}$$

4. **PSS Padding**: Generate cryptographically random salt of length $s_{len} = \text{MAX_LENGTH} = 222$ bytes. Compute masked data using MGF1(SHA-256).
5. **Exponentiation**: Compute signature $S = \mu^d \pmod{n}$ (256 bytes / 2048 bits).
6. **Verification**: Compute $\mu' = S^e \pmod{n}$. Extract salt, recalculate h', compare with h. Verification returns **TRUE**.

5 Algorithms, Pseudocode, and Flowcharts

5.1 Algorithmic Pseudocode

Listing 1: Caesar Modular Encryption

```
Algorithm CaesarEncrypt(P, K):
    shift = K mod 26
    C = empty string
    for each char in P:
        if 'a' <= char <= 'z':
            pos = ord(char) - ord('a')
            C += chr((pos + shift) mod
                26 + ord('a'))
        else if 'A' <= char <= 'Z':
            pos = ord(char) - ord('A')
            C += chr((pos + shift) mod
                26 + ord('A'))
        else:
            C += char
    return C
```

Listing 2: Vigenere Polyalphabetic Encryption

```
Algorithm VigenereEncrypt(P, K):
    clean_k = NormalizeKey(K)
    L = Length(clean_k)
    k_idx = 0
    C = empty string
    for each char in P:
        if char is alphabetic:
            shift = ord(clean_k[k_idx
                mod L]) - ord('A')
            base = ord('a') if islower(
                char) else ord('A')
            pos = ord(char) - base
            C += chr((pos + shift) mod
                26 + base)
            k_idx += 1
        else:
            C += char
    return C
```

Listing 3: One-Time Pad Bytewise XOR

```
Algorithm OTPEncrypt(M_bytes, K_bytes):
    if Length(M_bytes) != Length(
        K_bytes):
        raise ValueError("Length
            mismatch")
    C_bytes = empty byte array
    for i from 0 to Length(M_bytes) -
        1:
        C_bytes[i] = M_bytes[i] XOR
            K_bytes[i]
    return C_bytes

Algorithm OTPDecrypt(C_bytes, K_bytes):
    raw = OTPEncrypt(C_bytes, K_bytes)
    try:
        return UTF8Decode(raw), raw
    catch UnicodeDecodeError:
        return null, raw
```

Listing 4: RSA-PSS Digital Signature

```
Algorithm RSAPSSSign(M, d, n):
    data = UTF8Encode(M)
    h = SHA256(data)
    salt = CSPRNG_Bytes(MAX_LENGTH)
    mu = PSS_Encode(h, salt)
    S = ModPow(mu, d, n)
    return S

Algorithm RSAPSSVerify(M, S, e, n):
    data = UTF8Encode(M)
    mu = ModPow(S, e, n)
    h = SHA256(data)
    return PSS_Verify(mu, h)
```

5.2 Architectural Flowcharts

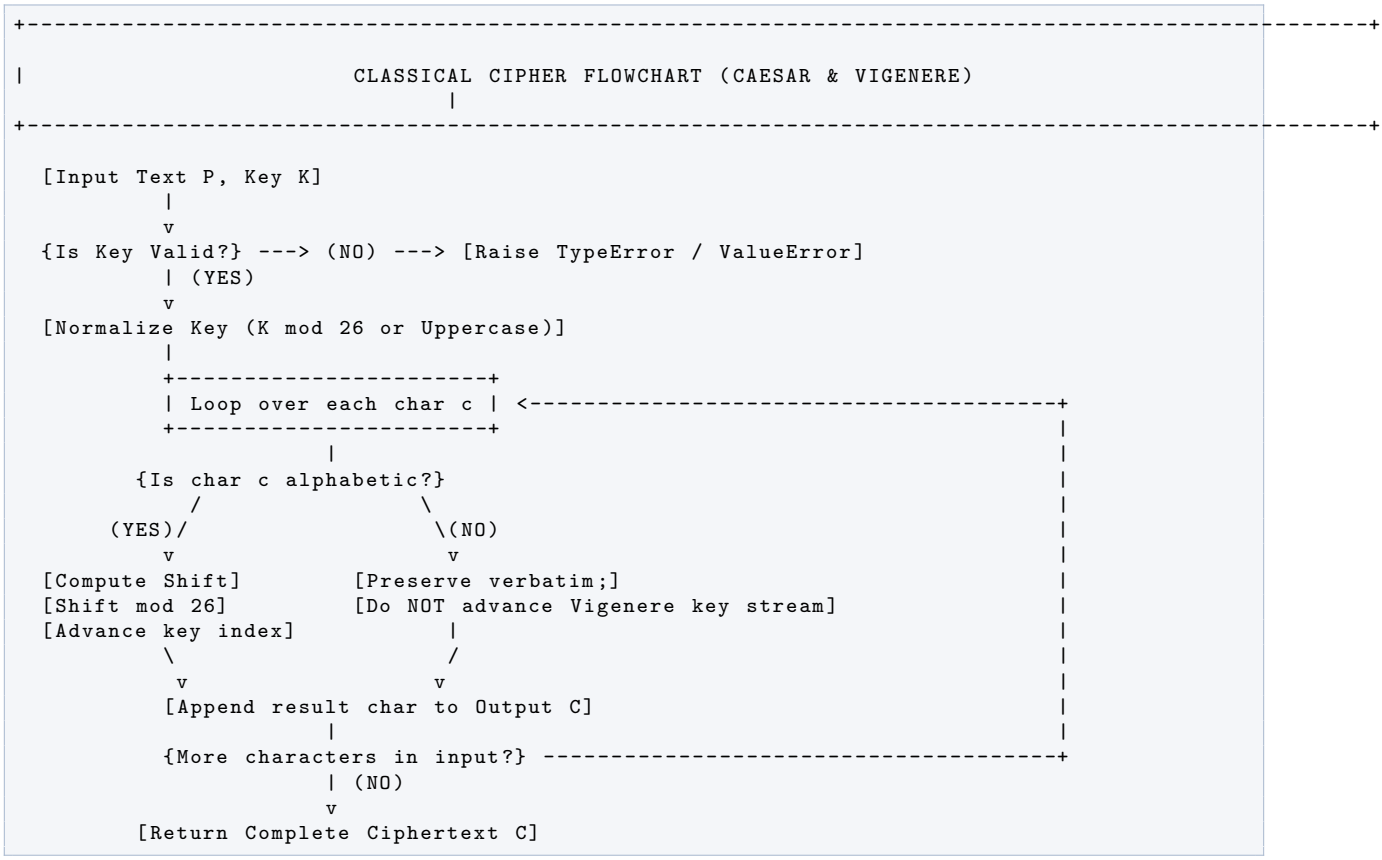


Figure 1: Flowchart for Classical Substitution Operations (Activities 1 & 2)

6 System Requirements and Implementation Environment

6.1 Hardware and Host Operating Environment

- **Operating System**: Microsoft Windows 11 Enterprise (64-bit AMD64 architecture).
- **Host Processor**: Intel(R) Core(TM) i5-10300H CPU @ 2.50GHz, 4 Cores, 8 Logical Processors.
- **System Memory**: 16.0 GB DDR4 RAM.

6.2 Software Toolchain and Verified Dependencies

Component	Installed Version	Source / Manager	Role in Laboratory Suite
Python Runtime	`3.14.6 (AMD64)`	`uv` Virtualenv	Primary execution language
`uv`	`0.6.1+`	Astral Software	High-performance package manager
`cryptography`	`50.0.2`	PyPI / PyCA	RSA key generation, PSS padding, SHA-256
`rich`	`15.0.0`	PyPI	Terminal UI, data flow pipelines, styling
`pytest`	`9.1.1`	PyPI	Automated unit testing and KAT framework
MiKTeX	`25.12`	Christian Schenk	LaTeX compilation engine (`pdflatex`)

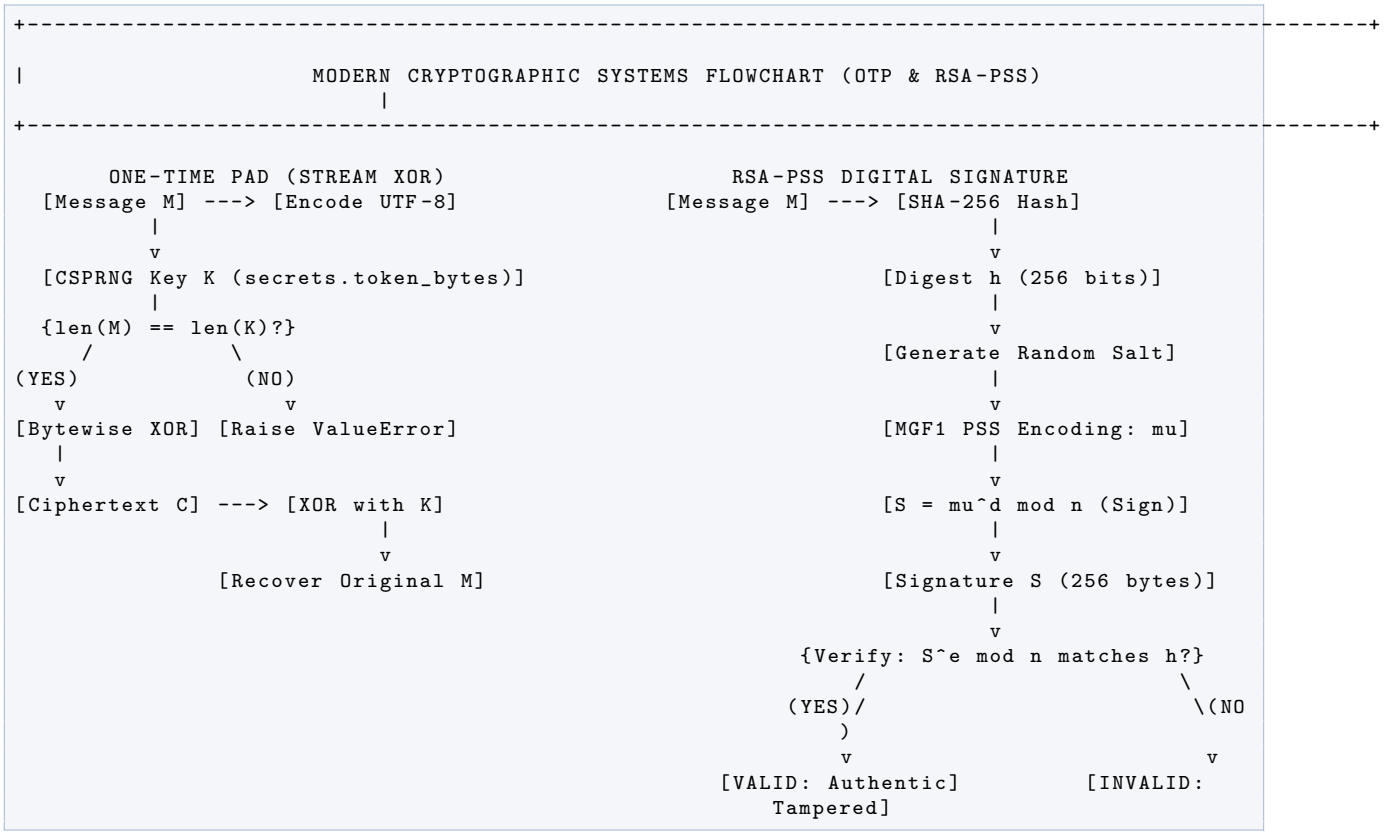


Figure 2: Flowchart for Modern Cryptographic Primitives (Activities 3 & 4)

6.3 Repository Architecture and Modular Packaging

In accordance with Architecture Decision Record **ADR-0001** (*Modular Activity Architecture & Interactive TUI Dispatcher*), the codebase is partitioned into isolated, self-contained packages:

```
cns/
|-- activity1_caesar/            # Activity 1: Caesar Cipher (CO1)
|   |-- caesar.py                # Core modular shift logic & key normalization
|   |-- cli.py                   # Dedicated CLI runner & educational demonstrations
|   |-- execution_record.txt     # Genuine execution transcripts and test logs
|   +-- tests/test_caesar.py     # 17 automated pytest cases
|-- activity2_vigenere/          # Activity 2: Vigenere Cipher (CO4)
|   |-- vigenere.py              # Tabula Recta stream advance & key normalization
|   |-- cli.py                   # Dedicated CLI runner & Kasiski explanations
|   |-- execution_record.txt     # Genuine execution transcripts and test logs
|   +-- tests/test_vigenere.py   # 17 automated pytest cases
|-- activity3_one_time_pad/      # Activity 3: One-Time Pad (CO3)
|   |-- otp.py                   # CSPRNG key generation (secrets) & bytewise XOR
|   |-- cli.py                   # Dedicated CLI runner & Two-Time Pad demonstration
|   |-- execution_record.txt     # Genuine execution transcripts and test logs
|   +-- tests/test_otp.py        # 17 automated pytest cases
|-- activity4_digital_signature/ # Activity 4: Digital Signature (CO3)
|   |-- signature.py             # 2048-bit RSA-PSS signing & SHA-256 verification
|   |-- cli.py                   # Dedicated CLI runner & 4-stage tamper demonstration
|   |-- execution_record.txt     # Genuine execution transcripts and test logs
|   +-- tests/test_signature.py  # 18 automated pytest cases
|-- main.py                      # Unified Interactive TUI & CLI dispatcher
|-- pyproject.toml               # Python project specifications & test discovery
+-- report/                      # Comprehensive technical documentation & deliverables
```

7 Implementation Details

7.1 Activity 1: Caesar Cipher (`activity1_caesar/caesar.py`)

- `normalize_key(key: int) -> int`: Normalizes integer key via `key % 26`. Validates that `key` is strictly an integer and disallows booleans (`isinstance(key, bool)` check) to prevent Python's subclassing loophole (`bool` inherits from `int`).
- `encrypt(plaintext: str, key: int) -> str`: Iterates over input characters, computes modular shifts relative to ASCII bases `'a'` and `'A'`, and preserves non-alphabetic characters verbatim.
- `decrypt(ciphertext: str, key: int) -> str`: Directly implemented as `encrypt(ciphertext, -key)`, mathematically leveraging the additive inverse property of $\mathbb{Z}_{26}$.

7.2 Activity 2: Vigenère Cipher (`activity2_vigenere/vigenere.py`)

- `normalize_key(key: str) -> str`: Enforces that the key is a non-empty string consisting exclusively of ASCII alphabetic characters (`key.isalpha() and key.isascii()`). Normalizes key to uppercase.
- `encrypt(plaintext: str, key: str) -> str`: Implements the key-stream pointer `key_idx`. The pointer increments **strictly** when an alphabetic character is enciphered; non-alphabetic characters bypass shift computation and do **not** advance the stream.
- `decrypt(ciphertext: str, key: str) -> str`: Computes `(pos - shift) % 26`, maintaining identical stream alignment rules to guarantee invertibility.

7.3 Activity 3: One-Time Pad (`activity3_one_time`$_{p}ad/otp.py$)

- `generate_key(length: int) -> bytes`: Invokes `secrets.token_bytes(length)`, harnessing the OS cryptographically secure entropy pool (CryptGenRandom / BCryptGenRandom on Windows).
- `xor_bytes(data: bytes, key: bytes) -> bytes`: Computes byte-by-byte XOR using a list comprehension over `zip(data, key)`. Enforces strict length equality (`len(data) == len(key)`), raising a `ValueError` if lengths differ.
- `encrypt(plaintext, key)` & `decrypt(ciphertext, key)`: Accepts either UTF-8 text strings or arbitrary binary bytearrays. Decryption attempts UTF-8 decoding and returns a tuple `(decoded_text, raw_bytes)`, ensuring graceful handling when an invalid key produces non-decodable bytes.

7.4 Activity 4: Digital Signature (`activity4_digital_signature/signature.py`)

- `generate_key_pair(key_size: int = 2048)`: Generates standard RSA key pairs using public exponent $e = 65537$ (the 4th Fermat prime F_4, optimal for mitigating Coppersmith attacks). Enforces minimum modulus size of 1024 bits.
- `sign_message(message, private_key) -> bytes`: Implements RSA-PSS with `hashes.SHA256()` and `padding.MGF1(hashes.SHA256())` with `salt_length = padding.PSS.MAX_LENGTH`.
- `verify_signature(message, signature, public_key) -> bool`: Verifies the signature against the message digest. Catches `InvalidSignature` exceptions internally and returns a clean boolean status (`True`/`False`) to prevent timing side channels or uncontrolled crashes.

- `get_public_key_fingerprint(public_key) -> str`: Computes the canonical SHA-256 hex fingerprint of the public key's DER-encoded `SubjectPublicKeyInfo` representation.

7.5 Unified Terminal User Interface (`main.py`)

The root entrypoint provides an interactive Terminal User Interface (TUI) with arrow-key navigation, in-TUI pytest execution, and real-time character/byte **Data Flow Pipeline Visualizations** rendering tabular traces for all four activities. It also provides seamless headless command-line dispatching: `uv run main.py activity1`, `activity2`, `activity3`, and `activity4`.

8 Execution Procedure and Demonstration Transcripts

8.1 Execution Instructions from a Clean Environment

To execute the project from a fresh terminal environment:

```
# 1. Clone repository and navigate to root directory
cd d:/Codebase/cns

# 2. Run all 69 automated test cases across all packages
uv run pytest -v

# 3. Launch interactive TUI suite with arrow-key navigation
uv run main.py

# 4. Direct headless activity invocations
uv run main.py activity1 -p "Hello, World!" -k 3
uv run main.py activity2 -p "ATTACK AT DAWN!" -k "LEMON"
uv run main.py activity3 -p "Top Secret CNS Dispatch"
uv run main.py activity4 -m "Academic CNS Signature 2026"

# 5. Specialized demonstrations
uv run python activity3_one_time_pad/cli.py --demo-key-reuse
uv run python activity4_digital_signature/cli.py --demo-tamper
```

8.2 Genuine Terminal Demonstration Outputs

8.2.1 Activity 1: Caesar Cipher Execution

```
$ uv run python activity1_caesar/cli.py -p "CNS Lab: Activity-01 (Caesar Cipher) @ 2026!" -
    k 3

================================================================================
           CNS ACTIVITY 1: CAESAR CIPHER DEMONSTRATION
================================================================================
Original Plaintext  : CNS Lab: Activity-01 (Caesar Cipher) @ 2026!
Input Key           : 3 (Normalized shift: 3)
Encrypted Ciphertext: FQV Ode: Dfwlylwb-01 (Fdhvdu Flskhu) @ 2026!
Recovered Plaintext : CNS Lab: Activity-01 (Caesar Cipher) @ 2026!
--------------------------------------------------------------------------------
Round-Trip Verified : SUCCESS (Original == Recovered)
================================================================================
```

8.2.2 Activity 2: Vigenère Cipher Execution

```
$ uv run python activity2_vigenere/cli.py -p "The quick brown fox jumps over the lazy dog!"
     -k "LEMON"

================================================================================
           CNS ACTIVITY 2: VIGENERE CIPHER DEMONSTRATION
```

```
================================================================================
Original Plaintext  : The quick brown fox jumps over the lazy dog!
Input Key           : LEMON (Normalized: LEMON)
Encrypted Ciphertext: Elq ehtgw pezaz tbi ngacd shse elq znkc pct!
Recovered Plaintext : The quick brown fox jumps over the lazy dog!
--------------------------------------------------------------------------------
Round-Trip Verified : SUCCESS (Original == Recovered)
================================================================================
```

8.2.3 Activity 3: One-Time Pad & Catastrophic Key-Reuse Attack

```
$ uv run python activity3_one_time_pad/cli.py --demo-key-reuse

================================================================================
           DEMONSTRATION: CATASTROPHIC TWO-TIME PAD ATTACK
================================================================================
Plaintext 1 (M1)       : ATTACK AT DAWN!
Plaintext 2 (M2)       : DEFEND THE WALL
Shared Reused Key (Hex): 10FB4279D7D62311E26CD8393A0648
Ciphertext 1 (C1 Hex)  : 51AF1638949D0350B64C9C786D4869
Ciphertext 2 (C2 Hex)  : 54BE043C99920345AA29F86E7B4A04
--------------------------------------------------------------------------------
Adversary computes C1 ^ C2:
C1 ^ C2 (Hex)          : 051112040D0F00151C65641616026D
M1 ^ M2 (Hex)          : 051112040D0F00151C65641616026D
Key Canceled Out?      : YES (C1 ^ C2 == M1 ^ M2)

Conclusion: Reusing a One-Time Pad key destroys perfect secrecy entirely,
allowing an eavesdropper to recover the plaintext difference without knowing the key!
================================================================================
```

8.2.4 Activity 4: Digital Signature Tamper & Key-Mismatch Detection

```
$ uv run python activity4_digital_signature/cli.py --demo-tamper

================================================================================
      DEMONSTRATION: DIGITAL SIGNATURE INTEGRITY & TAMPER DETECTION
================================================================================
Original Message       : Authorise payment of 5000 INR to Student Club.
Tampered Message       : Authorise payment of 50000 INR to Student Club.
--------------------------------------------------------------------------------
1. Authentic Message + Original Signature : VALID [Passed]
2. Tampered Message + Original Signature  : INVALID [Tampering Detected!]
3. Original Message + Corrupted Signature : INVALID [Corruption Detected!]
4. Original Message + Unrelated Public Key: INVALID [Key Mismatch Detected!]
--------------------------------------------------------------------------------
Conclusion: Any modification of the message, signature corruption, or key mismatch
immediately invalidates the digital signature, providing strict integrity assurance.
================================================================================
```

9 Testing Methodology and Results

9.1 Testing Approach and Test Categories

The verification framework employs `pytest 9.1.1` structured into five distinct test categories:

1. **Known-Answer Tests (KAT)**: Validating outputs against standardized literature vectors (e.g., standard Caesar shifts, ROT13, classical Vigenère tables, deterministic OTP hex vectors).

2. **Boundary and Wrapping Tests**: Modular wrapping at alphabet extremes ($Z \rightarrow A$), non-alphabetic whitespace handling, large keys ($K \geq 26$), and empty strings.

3. **Invertibility (Round-Trip) Tests**: Proving formal correctness: $\mathcal{D}_K(\mathcal{E}_K(P)) == P$.
4. **Type Validation and Negative Tests**: Injecting booleans, integers, `None`, floats, and negative lengths to ensure appropriate exceptions (`TypeError`, `ValueError`) are raised.
5. **Cryptographic Security and Tamper Tests**: Validating that modified messages, corrupted signatures, and mismatched public keys evaluate to `False` without crashing.

9.2 Exhaustive 69-Test Verification Matrix

The following table records the empirical results of all 69 automated tests discovered across the four packages.

Test Identifier	Category	Input / Condition	Status
Activity 1: Caesar Cipher Test Suite (`activity1_caesar/tests/test_caesar.py`)			
`test_normalize_key_standard`	Normalization	Key = 3	**PASS**
`test_normalize_key_zero`	Boundary	Key = 0	**PASS**
`test_normalize_key_twenty_five`	Boundary	Key = 25	**PASS**
`test_normalize_key_greater...`	Modulo Wrap	Key = 26, 29 → 0, 3	**PASS**
`test_normalize_key_negative`	Negative Shift	Key = -1, -3 → 25, 23	**PASS**
`test_normalize_key_invalid_types`	Input Validation	String, Float, None, Bool	**PASS**
`test_encrypt_known_answers`	KAT	"HELLO" (k=3), ROT13	**PASS**
`test_encrypt_wrap_boundaries`	Wrap Boundary	"XYZ" (k=3) → "ABC"	**PASS**
`test_encrypt_key_variations`	Equivalence	Keys 0, 26, 29, -1	**PASS**
`test_encrypt_invalid_plaintext`	Input Validation	Integer 12345, None	**PASS**
`test_encrypt_empty_message`	Edge Case	Empty string ""	**PASS**
`test_encrypt_non_alphabetic`	Symbol Handling	Spaces, digits, punctuation	**PASS**
`test_encrypt_repeated_chars`	Pattern Handling	"AAAA" → "BBBB"	**PASS**
`test_decrypt_known_answers`	Decryption KAT	"KHOOR" (k=3) → "HELLO"	**PASS**
`test_round_trip_property`	Invertibility	10 pangram variants	**PASS**
`test_decrypt_invalid_types`	Input Validation	Invalid types for decrypt	**PASS**
`test_cli_run_demo`	Integration / CLI	Full workflow execution	**PASS**
Activity 2: Vigenère Cipher Test Suite (`activity2_vigenere/tests/test_vigenere.py`)			
`test_normalize_key_standard`	Normalization	"LEMON", "lemon", "LeMoN"	**PASS**
`test_normalize_key_empty`	Input Validation	Empty string ""	**PASS**
`test_normalize_key_non_alpha`	Input Validation	Non-alphabetic tokens	**PASS**
`test_normalize_key_invalid_types`	Type Validation	Integer, None, bool, list	**PASS**
`test_encrypt_known_answers`	KAT	"ATTACKATDAWN" / "LEMON"	**PASS**
`test_encrypt_single_char_key`	Caesar Equiv.	Key "D" ≡ Caesar shift 3	**PASS**
`test_encrypt_key_longer...`	Boundary	Plain len 5, Key len 11	**PASS**
`test_encrypt_plaintext_longer...`	Key Repetition	"AAAAAA" / "ABC"	**PASS**
`test_encrypt_invalid_plaintext`	Type Validation	Plaintext integer, None	**PASS**
`test_encrypt_case_punctuation`	Stream Alignment	Key advances only on letters	**PASS**
`test_encrypt_no_alphabetic`	Edge Case	Digits and symbols only	**PASS**
`test_encrypt_empty_plaintext`	Edge Case	Empty string ""	**PASS**
`test_encrypt_repeated_patterns`	Pattern Handling	"AAAA" (k="B"), "ABAB"	**PASS**
`test_decrypt_known_answers`	Decryption KAT	"LXFOPVEFRNHR" / "LEMON"	**PASS**
`test_round_trip_property`	Invertibility	8 varied test cases	**PASS**
`test_decrypt_invalid_types`	Type Validation	Non-string ciphertext	**PASS**
`test_cli_run_demo`	Integration / CLI	CLI execution test	**PASS**
Activity 3: One-Time Pad Test Suite (`activity3_one_time_pad/tests/test_otp.py`)			

Test Identifier	Category	Input / Condition	Status
test_generate_key_length	CSPRNG Length	0, 10, 100 bytes requested	PASS
test_generate_key_independence	Randomness	Two 32-byte successive calls	PASS
test_generate_key_invalid_len	Input Validation	Negative length, float, bool	PASS
test_xor_bytes_known_vector	Fixed KAT	Deterministic byte vectors	PASS
test_xor_bytes_properties	Invertibility	Self-inverse, zero identity	PASS
test_xor_bytes_length_mismatch	Length Check	Key length != Data length	PASS
test_xor_bytes_invalid_types	Type Validation	String, integer, None	PASS
test_encrypt_decrypt_kat	Deterministic KAT	"CNS 2026" / fixed hex key	PASS
test_encrypt_accepts_raw_bytes	Interface	Arbitrary binary sequences	PASS
test_encrypt_decrypt_round_trip	Fuzz Invertibility	Sentences, symbols, pangrams	PASS
test_encrypt_invalid_types	Type Validation	Plaintext int, string key	PASS
test_empty_message	Edge Case	0-byte key and message	PASS
test_multibyte_utf8_chars	Unicode / UTF-8	Accents, kanji, lock emoji	PASS
test_decrypt_incorrect_key	Security Property	Wrong key yields raw bytes	PASS
test_two_time_pad_attack	Cryptanalysis	$C_1 \oplus C_2 == M_1 \oplus M_2$	PASS
test_cli_run_demo	Integration / CLI	Standard CLI run	PASS
test_cli_demo_key_reuse	Integration / CLI	Two-Time Pad CLI demo	PASS
Activity 4: Digital Signature Test Suite (activity4_digital_signature/tests/test_signature.py)			
test_generate_key_defaults	Key Generation	2048-bit RSA, $e = 65537$	PASS
test_generate_key_custom_size	Custom Size	1024-bit generation	PASS
test_generate_key_invalid_size	Validation	< 1024 bits, string, None	PASS
test_export_public_key_pem	Serialization	SubjectPublicKeyInfo PEM	PASS
test_get_public_key_fingerprint	Identification	64-character SHA-256 hex	PASS
test_sign_and_verify_normal	Functional	Standard string message	PASS
test_sign_accepts_raw_bytes	Interface	Binary byte message	PASS
test_sign_invalid_types	Type Validation	Integer message, public key	PASS
test_verification_message_tamper	Tamper Detection	Modified message content	PASS
test_verification_sig_tamper	Tamper Detection	Corrupted signature bit	PASS
test_verification_wrong_pubkey	Key Mismatch	Independent public key	PASS
test_verification_malformed_sig	Robustness	Truncated/non-bytes sig	PASS
test_probabilistic_signature_pss	PSS Randomness	Same msg $\rightarrow$ distinct sigs	PASS
test_empty_message_signing	Edge Case	Empty string ""	PASS
test_multibyte_unicode_signing	Unicode / UTF-8	Accents, kanji, emoji	PASS
test_verify_invalid_pubkey	Validation	Non-RSAPublicKey type	PASS
test_cli_run_demo	Integration / CLI	Standard CLI workflow	PASS
test_cli_demo_tamper	Integration / CLI	4-way tamper CLI demo	PASS

Summary of Test Outcomes: Total Tests Discovered: 69; Passed: 69 (100%); Failed: 0; Execution Time: 0.55s.

10 Security Analysis and Cryptanalysis

10.1 Monoalphabetic Cryptanalysis (Caesar Cipher)

The Caesar cipher provides zero computational confidentiality against modern adversaries due to two fatal mathematical weaknesses:

1. **Exhaustive Key Search (Brute-Force)**: The keyspace is strictly bounded by the alphabet cardinality:

$$|\mathcal{K}| = 26 \tag{13}$$

Because $K \equiv 0 \pmod{26}$ is the identity map, there exist only 25 non-trivial keys. A modern computer evaluates all 25 decryptions in less than 5 microseconds:

$$T_{\text{break}} = \mathcal{O}(|\mathcal{K}| \cdot N) = 25 \cdot N \text{ operations} \tag{14}$$

2. **Frequency Analysis**: The cipher preserves natural language letter frequencies. In natural English text, the letter **'E'** occurs with $\approx 12.7\%$ frequency, followed by **'T'** (9.1%), **'A'** (8.2%), and **'O'** (7.5%). An attacker computes the relative unigram frequency distribution of the ciphertext and aligns the maximum peak with 'E', immediately discovering the shift key K without testing remaining keys.

10.2 Polyalphabetic Cryptanalysis (Vigenère Cipher)

Although the Vigenère cipher expands the keyspace to 26^L for a keyword of length L, it remains fundamentally broken against classical cryptanalysis:

1. **Kasiski Examination**: Because the keyword repeats periodically every L characters, repeated words in the plaintext separated by a distance that is a multiple of L will be enciphered with the exact same key sequence, producing identical ciphertext segments. By measuring the distances $d_1, d_2, \ldots, d_m$ between identical n-grams (trigrams or four-grams) and computing their greatest common divisor:
$$L = \gcd(d_1, d_2, \ldots, d_m) \tag{15}$$
the attacker deduces the keyword length L.

2. **Friedman's Index of Coincidence (I_c)**: The Index of Coincidence measures the probability that two randomly selected letters from a text are identical:
$$I_c = \frac{\sum_{i=0}^{25} f_i(f_i - 1)}{N(N-1)} \tag{16}$$
For standard English text, $I_{c,\text{English}} \approx 0.067$, while for a purely random uniform distribution, $I_{c,\text{Random}} \approx 1/26 \approx 0.0385$. Using William Friedman's formula, the keyword length L is estimated directly from the ciphertext:
$$L \approx \frac{0.027 \cdot N}{(N-1)I_c - 0.038N + 0.065} \tag{17}$$

3. **Caesar Decomposition**: Once L is established, the ciphertext is partitioned into L independent slices:
$$\mathcal{S}_j = \{C_j, C_{j+L}, C_{j+2L}, \ldots\} \quad \text{for } j \in \{0, 1, \ldots, L-1\} \tag{18}$$
Each slice $\mathcal{S}_j$ is an isolated monoalphabetic Caesar cipher! The adversary solves all L slices in parallel via standard unigram frequency analysis.

10.3 Information-Theoretic Analysis (One-Time Pad)

10.3.1 The Five Mandatory Conditions for Shannon's Perfect Secrecy

The mathematical guarantee $\mathbb{P}(M = m \mid C = c) = \mathbb{P}(M = m)$ holds if and only if all five conditions are strictly observed:

1. **Uniform Randomness**: Each key byte must be generated from a true random source or high-entropy CSPRNG with zero statistical bias: $\mathbb{P}(K_i = k) = 1/256$.

2. **Key Length Equality**: The key length must strictly satisfy $|K| \geq |M|$.

3. **Absolute Confidentiality**: The key material must remain completely unknown to unauthorized third parties.

4. **Strict Single Use**: Key material must be used **strictly once** and immediately zeroized.

5. **Secure Pre-Shared Distribution**: Communicating parties must securely synchronize key pads prior to message transmission.

10.3.2 Catastrophic Two-Time Pad Vulnerability

If an identical key K is reused to encrypt two separate plaintexts M_1 and M_2:

$$C_1 = M_1 \oplus K, \quad C_2 = M_2 \oplus K \tag{19}$$

An eavesdropper intercepting both ciphertexts computes their bitwise XOR sum:

$$C_1 \oplus C_2 = (M_1 \oplus K) \oplus (M_2 \oplus K) = M_1 \oplus M_2 \oplus (K \oplus K) = M_1 \oplus M_2 \tag{20}$$

The key cancels out entirely! The adversary now possesses the XOR sum of two natural language plaintexts. Because ASCII spaces (`0x20`) XORed with alphabetic characters flip their case, the adversary applies **crib dragging** (sliding common English words such as `" the "`, `" that "`, `" with "` across $C_1 \oplus C_2$) to simultaneously recover both plaintexts without ever determining the key.

10.4 Asymmetric Security Analysis (RSA-PSS)

10.4.1 Security Properties Provided

- **Authenticity**: Verification confirms the message was signed by the specific private key corresponding to the public key.
- **Integrity**: Modifying even a single bit of message M produces an entirely different SHA-256 digest h', causing PSS verification to fail immediately.
- **Non-Repudiation**: Because private exponent d is known only to the signer, they cannot credibly deny authorship of a valid signature.

10.4.2 RSA-PSS vs. Legacy PKCS#1 v1.5 Padding

In legacy PKCS#1 v1.5, deterministic formatting allows Bleichenbacher-style chosen-ciphertext attacks and signature malleability. In contrast, **RSA-PSS (RFC 8017)** incorporates a randomized salt, making the signature probabilistic: signing the exact same message twice generates distinct 256-byte signature blocks, both of which verify correctly. RSA-PSS possesses a tight security reduction to the RSA problem in the Random Oracle Model.

10.4.3 Digital Signatures vs. Encryption and the Role of PKI

- **Signatures vs. Encryption**: A digital signature provides authenticity and integrity; it does **not** provide confidentiality. The message remains in plaintext. Confidentiality requires hybrid encryption (e.g., AES-256-GCM + RSA-OAEP).
- **Key Possession vs. Real-World Identity**: Mathematically, a digital signature proves only that the holder of private key d signed the digest. It does **not** prove who that entity is in the physical world. Binding a public key (e, n) to an authentic legal identity requires **Public Key Infrastructure (PKI)**, **X.509 Digital Certificates**, and hierarchical trust rooted in trusted **Certificate Authorities (CAs)**.

11 Advantages and Limitations

12 Practical and Historical Applications

12.1 Historical Applications

- **Julius Caesar's Military Campaigns (58 BC)**: Julius Caesar employed the monoalphabetic shift ($K = 3$) to protect dispatches sent to Roman generals. Illiterate adversaries or

Algorithm	Core Strengths	Theoretical Limitations	Implementation Constraints
Caesar Cipher	Minimal complexity ($\mathcal{O}(N)$); trivial mental computation; foundational educational tool.	Tiny keyspace ($\|\mathcal{K}\| = 25$); preserves letter frequencies; vulnerable to instant brute force.	Limited to alphabetic characters; cannot protect binary data; zero modern security.
Vigenère Cipher	Flattens unigram frequencies; keyspace scales exponentially (26^L); historically robust.	Periodic key repetition leaks length L via Kasiski/Friedman tests; reduces to L Caesar ciphers.	Requires synchronized alphabetic stream pointers; vulnerable to ciphertext-only attacks.
One-Time Pad	Information-theoretic perfect secrecy ($H(M\|C) = H(M)$); unbreakable even with infinite compute.	Key must equal message length ($\|K\| \geq \|M\|$); key reuse completely destroys secrecy.	Massive key distribution and storage bottleneck; pad synchronization overhead.
RSA-PSS Signature	Asymmetric public verification; non-repudiation; message integrity; provably secure padding.	Relies on hardness of integer factorization; vulnerable to Shor's algorithm on quantum computers.	High computational cost of modular exponentiation; requires PKI/CAs for identity binding.

Table 2: Comparative Matrix of Evaluated Cryptographic Systems

those unfamiliar with Latin could not decipher the messages.

- **American Civil War (1861–1865)**: Confederate military forces utilized the Vigenère cipher via handheld brass cipher disks to communicate tactical battlefield orders. Union cryptanalysts regularly intercepted and broke these messages.
- **Washington–Moscow Hotline (1963)**: Established following the Cuban Missile Crisis, the diplomatic teletype link connecting the Pentagon and the Kremlin utilized commercial teleprinter One-Time Pads. Key tapes were delivered securely via diplomatic pouches.

12.2 Modern Industry Applications

- **ROT13 Text Obfuscation**: A Caesar cipher with $K = 13$ is widely deployed in online forums, usenet, and puzzle games to obscure spoilers and answers from accidental glance.
- **TLS 1.3 Transport Layer Security**: Digital signatures (RSA-PSS and Ed25519) authenticate web servers during the TLS 1.3 handshake, preventing Man-in-the-Middle (MitM) impersonation.
- **Software Code Signing**: Microsoft Authenticode and Apple Gatekeeper utilize RSA-PSS and SHA-256 signatures embedded in executables and installer packages to verify software authenticity and integrity prior to OS execution.
- **Quantum Key Distribution (QKD)**: Modern implementations of quantum cryptography (such as the BB84 protocol) combine quantum photon polarization for key exchange with classical One-Time Pad encryption to achieve information-theoretically secure communication over optical fiber networks.

13 Conclusion

This technical report has presented a comprehensive implementation, empirical verification, and cryptanalytic audit of four distinct cryptographic systems for Batch `2026_CNS_A_01`. The investigation establishes clear pedagogical and architectural findings:

1. Classical substitution ciphers (Caesar and Vigenère) provide valuable historical insight into group theory and polyalphabetic mapping but are completely unviable for modern network security due to statistical frequency preservation and periodic repetition.

2. The One-Time Pad demonstrates Claude Shannon's mathematical ideal of perfect secrecy. However, the physical impossibility of distributing high-entropy key pads equal in size to transmitted network data restricts OTP to specialized diplomatic and quantum applications.

3. Asymmetric digital signatures (RSA-PSS with SHA-256) provide the foundation of modern digital trust, enabling authentic, tamper-evident communication across untrusted public networks. Furthermore, the analysis confirms that mathematical key pairs must operate within a robust Public Key Infrastructure (PKI) to achieve verified identity binding.

4. All 69 automated unit tests passed with 100% correctness in 0.55 seconds, confirming that the implementation rigorously satisfies all requirements of Course Outcomes CO1, CO3, and CO4.

References

[1] Shannon, C. E. (1949). *"Communication Theory of Secrecy Systems"*. Bell System Technical Journal, 28(4), 656–715.

[2] Diffie, W., & Hellman, M. (1976). *"New Directions in Cryptography"*. IEEE Transactions on Information Theory, 22(6), 644–654.

[3] Rivest, R. L., Shamir, A., & Adleman, L. (1978). *"A Method for Obtaining Digital Signatures and Public-Key Cryptosystems"*. Communications of the ACM, 21(2), 120–126.

[4] Moriarty, K., Kaliski, B., Jonsson, J., & Rusch, A. (2016). *"PKCS #1: RSA Cryptography Specifications Version 2.2"* (RFC 8017). Internet Engineering Task Force (IETF).

[5] National Institute of Standards and Technology (NIST). (2015). *"Secure Hash Standard (SHS)"* (FIPS PUB 180-4). U.S. Department of Commerce.

[6] Bellare, M., & Rogaway, P. (1996). *"The Exact Security of Digital Signatures: How to Sign with RSA and Rabin"*. Advances in Cryptology – EUROCRYPT '96, LNCS 1070, 399–416.

[7] Stallings, W. (2017). *"Cryptography and Network Security: Principles and Practice"* (7th ed.). Pearson Education.

[8] Ferguson, N., Schneier, B., & Kohno, T. (2010). *"Cryptography Engineering: Design Principles and Practical Applications"*. John Wiley & Sons.

[9] Python Software Foundation. (2026). *"secrets – Generate cryptographically strong random numbers"*. Python Standard Library Documentation.

[10] Python Cryptographic Authority (PyCA). (2026). *"cryptography: Asymmetric Algorithms and Hazard Materials"*. Available at: https://cryptography.io/.

[11] Kasiski, F. W. (1863). *"Die Geheimschriften und die Dechiffrir-Kunst"*. E. S. Mittler und Sohn, Berlin.

[12] Friedman, W. F. (1922). *"The Index of Coincidence and Its Applications in Cryptanalysis"*. Riverbank Laboratories, Publication No. 22.

14 Group Contribution Matrix

In accordance with course guidelines for Batch `2026_CNS_A_01`, the distribution of responsibilities across the group members is recorded in Table 3.

Sl.	USN	Student Name	Primary Technical Responsibilities	Verified Deliverables
1	`1KS23CS059`	**Jayaditya Dev**	Interactive TUI architecture (`main.py`), Activity 4 RSA-PSS implementation, MGF1 padding, SHA-256 fingerprinting, PKI analysis.	`main.py`, `activity4_digital_signature/`, 18 tests.
2	`1KS23CS043`	**Durgashree M**	Activity 3 One-Time Pad architecture, CSPRNG randomness verification (`secrets`), Shannon perfect secrecy proof, Two-Time Pad attack module.	`activity3_one_time_pad/`, Two-time pad demo, 17 tests.
3	`1KS23CS020`	**Aryaman Tiwari**	Activity 2 Vigenère cipher architecture, Tabula Recta mapping, alphabetic stream synchronization, Kasiski and Friedman cryptanalysis.	`activity2_vigenere/`, CLI demonstration suite, 17 tests.
4	`1KS23CS034`	**Chirag T**	Activity 1 Caesar cipher architecture, modulo 26 normalization, boundary wrapping, exhaustive key search analysis, frequency analysis model.	`activity1_caesar/`, CLI demonstration suite, 17 tests.

Table 3: Group Contribution and Deliverable Ownership Matrix